

课程笔记

Claude Code 在大型代码库中的工作原理：最佳实践与入门指南

五道口纳什 & Codex

2026 年 6 月 3 日



视频作者/频道: AI 随风随风

发布日期: 2026-05-19

视频时长: 00:17:57

视频链接: https://www.bilibili.com/video/BV1gCLq6YEW4/?spm_id_from=333.337.search-card.all.click&vd_source=6457e444c35aa58365caef6c074dd705

目录

1 为什么大型代码库需要 Harness	2
1.1 问题入口：模型能力与工程环境的差距	2
1.2 Harness 的基本含义	2
1.3 组件总览：CLAUDE.md、Hooks、Skills、Plugins、LSP、MCP 与 Subagents	3
1.4 本章小结	5
2 Harness 组件如何分层工作	5
2.1 CLAUDE.md：全局规则与模块级规则	5
2.2 Hooks：把稳定动作变成自动触发	7
2.3 Skills：按需加载的专业流程	8
2.4 Plugins：团队级分发方式	8
2.5 MCP 与 CIAI：连接外部业务系统	9
2.6 本章小结	10
3 Code Map：让 Claude Code 找得到代码	10
3.1 默认搜索流程的成本	10
3.2 索引类比：从全量扫描到模块定位	11
3.3 CODE_MAP.md、根目录 CLAUDE.md 与子目录 CLAUDE.md 的协同	12
3.4 模型升级后的配置复查	13
3.5 本章小结	14
4 实施路线图	14
4.1 第一阶段：CLAUDE.md、CODE_MAP.md、模块级规则	15
4.2 第二阶段：Hooks 与 Skills 的沉淀	15
4.3 第三阶段：Plugins、MCP 与 Subagents	16
4.4 本章小结	16
5 总结与延伸	17
5.1 讲者的收束观点	17
5.2 跨工具迁移：为什么这些做法适用于多种 AI 编程工具	17
5.3 实践检查清单	18

1 为什么大型代码库需要 Harness

1.1 问题入口：模型能力与工程环境的差距

这段视频的入口很直接：讲者在 00:00:00--00:00:16 提到，Claude 最近发布了一篇关于在大型项目中使用 Claude Code 的最佳实践文章，里面的经验可以迁移到多种 AI 编程工具。这个开场给出了本章的核心问题：大型代码库中的失败，往往发生在模型进入真实工程环境之后。模型会写代码、解释代码、修改文件，可是它还需要知道项目规约、模块边界、业务系统、检索路径、团队沉淀下来的陷阱，以及哪些动作应当自动执行。

在小型项目里，AI 编程工具可以靠一次全局搜索、几段上下文和用户口头说明完成任务。大型代码库的情况尖锐得多：文件数量大，目录层级深，模块之间有隐含依赖，命名里带有历史债，业务规则常常散在文档、工单、配置和代码注释里。模型一旦缺少这些环境信息，就会把上下文窗口消耗在猜路径、重复搜索和修补误判上。结果看起来像“模型不够聪明”，真实原因常常是工程环境没有把它带到正确的位置。

可以用一个粗略的工程等式理解这个差距：

$$\text{有效产出} \approx \text{模型能力} \times \text{上下文质量} \times \text{导航效率} \times \text{流程稳定性}$$

- **模型能力**：基础模型的推理、编码、阅读和规划能力。
- **上下文质量**：项目规范、已知坑点、模块规则、业务约束进入模型上下文的质量。
- **导航效率**：模型从任务描述定位到相关文件、函数、类型和依赖的速度。
- **流程稳定性**：格式化、检查、复盘、分工、外部系统读取等动作能否稳定复现。

这个式子是一条工程经验：任一环节过低，最终产出都会明显缩水。大型代码库需要 Harness，原因就在这里。单纯升级模型，无法自动补齐项目知识、自动化动作、语义导航和团队级分发。后续章节会继续讨论 CODE_MAP.md，它承担的是代码地图和模块入口索引职责。

1.2 Harness 的基本含义

讲者在 00:00:21--00:00:44 把成败因素从“模型本身”推向 Harness，也可以译作“鞍具生态”。这个比喻很形象：马有力量，鞍具、缰绳和马镫决定骑手能否稳定控制方向、速度和姿态。对应到 Claude Code，大模型负责推理与生成，Harness 负责把它放进可控、可检索、可复用、可分发的工程环境。

Harness 的工作定义

Harness 指围绕 Claude Code 构建的一组工程化支撑系统：上下文文件提供规则，Hooks 自动触发生命周期动作，Skills 固化专用流程，Plugins 负责团队分发，LSP 提供代码语义导航，MCP 连接外部系统，Subagents 承担专门任务或并行任务。

这个定义要落在具体机制上。一个好的 Harness 至少解决四类问题。第一，**规则进入上下文**：模型需要自动读取项目级与模块级约束，减少用户每次重复解释。第二，**动作自动触发**：代码写完后应该检查、格式化、总结经验，不能每次靠人工提醒。第三，**知识按需加载**：复杂流程应封装成 Skills，在任务触发时加载，避免常驻上下文膨胀。第四，**系统连接与分工**：大型公司里的工单、产品系统、代码导航、团队插件和子代理协作，都需要稳定的接口与分发方式。

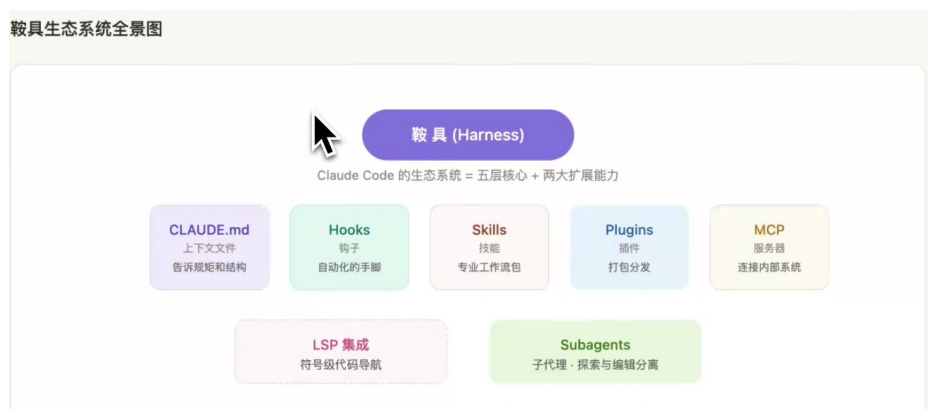


图 1: Harness 全景图展示 CLAUDE.md、Hooks、Skills、Plugins、MCP、LSP 与 Subagents 共同组成的 Claude Code 工程生态。¹

图 F1 的位置对应 00:00:30--00:00:50 的讲解窗口。它适合在 Harness 定义之后出现，因为读者已经知道“生态环境”这个词指向哪些实物：文件、脚本、协议、插件和代理。没有这些具体部件，Harness 会变成一句漂亮口号；有了这些部件，读者才能看出它怎样改变 Claude Code 在大型仓库里的工作路径。

1.3 组件总览：CLAUDE.md、Hooks、Skills、Plugins、LSP、MCP 与 Subagents

讲者在 00:00:49--00:01:04 给出了组件清单：五个常用部分是 CLAUDE.md、Hooks、Skills、Plugins、MCP；两个扩展部分是 LSP 与 Subagents。这个清单很重要，因为它把“让 AI 编程工具更好用”拆成了可操作的工程对象。每个对象有自己的职责边界，混在一起会导致上下文污染、流程重复和团队分发困难。

组件	是什么	何时加载	最适用场景	常见误区
CLAUDE.md	Claude 自动读取的上下文文件	每次会话	项目特定约定、代码库知识	把应放在 Skill 中的可复用专业知识放进来
Hooks	在关键时刻运行的脚本	由事件触发	自动化一致行为、捕获会话学习	用提示词做应该自动运行的事情
Skills	针对特定任务类型的打包指令	按需加载，当相关时	跨会话和项目的可复用专业知识	把所有内容都加载到 CLAUDE.md 中
Plugins	打包的技能、钩子和 MCP 配置	配置后始终可用	在组织内分发有效的设置	让好的设置停留在部落知识中
LSP*	通过语言特定服务器提供实时代码智能	配置后始终可用	符号级导航和类型语言中的自动错误检测	认为它是自动启用的
MCP 服务器	连接外部工具和数据的接口	配置后始终可用	让 Claude 访问无法直接触及的内部工具	在基础配置还没搞好之前就构建 MCP 连接
Subagents*	针对特定任务的独立 Claude 实例	被调用时	将探索与编辑分离、并行工作	在同一会话中运行探索和编辑

图 2: 组件速查表对比 CLAUDE.md、Hooks、Skills、Plugins、LSP、MCP 与 Subagents 的加载时机、适用场景和常见误区。²

CLAUDE.md：常驻上下文入口。在 00:01:08--00:01:38，讲者强调 CLAUDE.md 会在每一次对话里加载进上下文。这里先把它作为组件清单项交代：它让项目级约束稳定进入 Claude Code 上下

¹ 视频画面时间区间：00:00:30–00:00:50

² 视频画面时间区间：00:00:55–00:01:25

文；完整的根目录与模块级分层规则放到下一章展开。

CLAUDE.md 的常见误区

常见误区是把所有经验都写进 CLAUDE.md。本章只建立组件边界：常驻规则、按需流程和团队分发要分层处理；下一章会展开具体放置方式。

Hooks：生命周期事件触发的自动动作。在 00:01:40--00:02:55，讲者把 Hooks 解释为 Claude Code 对话生命周期中的事件触发点。发起对话可以触发一个 Hook，完成对话也可以触发一个 Hook；Hook 对应的动作可以是自定义代码。常见用途包括代码格式化、代码简化、检查、可视化，以及更关键的“捕获会话学习”：对话结束后总结本轮协作中值得沉淀的经验，再把它转成 Skill 或写入 CLAUDE.md 约束。这样做的实际效果是减少同类错误的重复出现。

Skills：按需加载的专业流程。在 00:03:01--00:03:21，讲者说 Skills 可以把专用知识和固定流程封装起来，由模型按任务需要加载。它解决的是“同一类任务反复做，结果还要尽量稳定”的问题。例如字幕精修、PDF 课程笔记生成、代码评审、测试驱动修复，都可以写成 Skill。Skill 的价值来自流程确定性：输入相似时，模型会沿着同一套步骤检查、写作、验证和交付。

Plugins：团队级分发层。在 00:03:21--00:03:49，讲者特别强调 Plugins 的分发价值。团队可以把 Hooks、Skills、Subagents 以及相关配置打包成插件，通过一条安装指令分发给不同成员。这个机制类似 Java 私有库或 NPM 包管理：单个开发者沉淀的流程，可以变成团队可安装、可升级、可治理的公共能力。大型代码库里的 Harness 需要这一层，因为只靠个人配置很难保持团队一致性。

LSP：让模型拥有语义导航和诊断能力。在 00:03:50--00:04:22，讲者用代码跳转说明 LSP 的作用。假设当前代码调用了类似 `selectList` 的方法；没有 LSP 时，Claude Code 可能先找文件，再找方法，再读实现，路径长且容易误判。安装 LSP 后，它可以像开发者在 IDE 里“跳转到定义”一样定位具体代码，还能获得自动错误检测。LSP 的完整名称是 Language Server Protocol，它让编辑器或代理通过统一协议获得定义跳转、引用查找、类型信息和诊断结果。

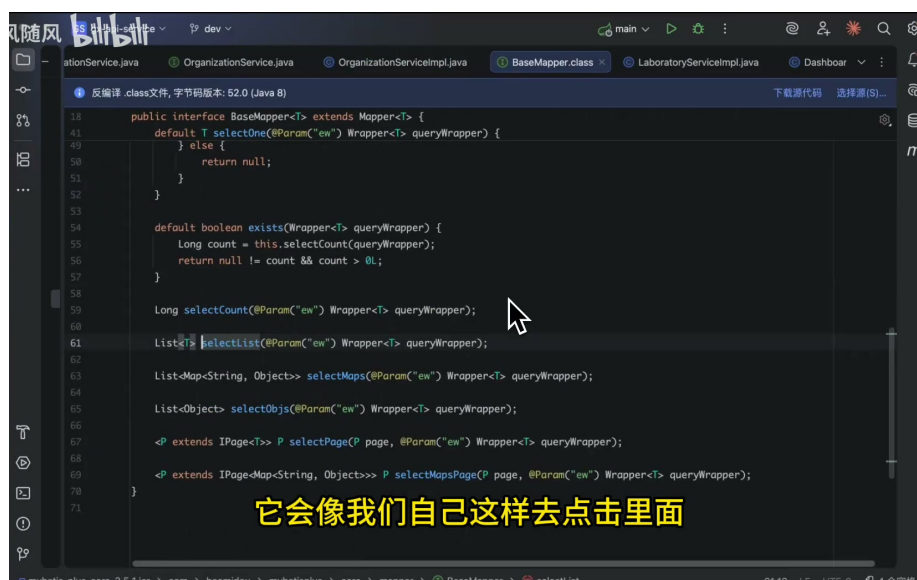


图 3: LSP 导航示例显示编辑器从 `selectList` 调用跳转到 `BaseMapper.class` 中的具体定义位置。

3

³视频画面时间区间：00:03:55--00:04:25

LSP 与普通文本搜索的区别

LSP 的关键价值是“语义导航”。普通文本搜索看到的是字符串；LSP 看到的是符号、类型、定义、引用和诊断。对大型代码库来说，这相当于从翻目录升级到使用专业索引。

讲者在 00:04:23--00:04:52 还给出安装线索：先安装官方 Plugins 相关能力，再进入 Discovery 搜索 LSP；如果项目是 Java，可以选择 JD TLS；安装到全局后，多个项目都能受益。这里的重点是工程习惯：LSP 应作为大型仓库的基础设施配置，而非临时补救工具。

MCP：连接外部业务系统的协议层。在 00:04:55--00:05:20，讲者把场景放到大型公司：工单系统、产品系统、内部平台都可能保存着任务所需信息。MCP 负责把工单、需求、验证结果和回填动作纳入授权流程。CIAI 及权限、审计、数据边界风险放到下一章集中展开。

Subagents：专门化与并行化的执行单元。Subagents 在本章属于扩展组件。它们可以理解多个独立 Claude Code 实例，各自承担清晰职责：一个读代码，一个写测试，一个找图，一个做一致性检查。对大型代码库而言，Subagents 的意义在于隔离上下文和拆分工作，避免单个会话同时背负过多任务细节。后续章节会把它放进实施路线图中讨论。

1.4 本章小结

本章建立了一个判断框架：大型代码库里的 Claude Code 效果，取决于模型能力与工程环境的乘积。视频在 00:00:21--00:00:44 用 Harness 比喻说明，模型需要被放进一个能提供规则、导航、自动化、分发和系统连接的生态中。随后 00:00:49--00:05:20 的组件清单把这个生态拆成七个可执行对象：CLAUDE.md 负责常驻规则，Hooks 负责生命周期自动动作，Skills 负责按需流程，Plugins 负责团队分发，LSP 负责语义导航与诊断，MCP 负责外部系统连接，Subagents 负责专门化与并行化。

读者应带走的结论很硬：在大型代码库里，AI 编程工具的瓶颈经常出现在环境设计上。缺少 Harness，模型会反复猜路径、漏掉约束、重复犯错、依赖人工转述；Harness 建好之后，Claude Code 才更接近一个能进入真实工程流水线的开发代理。下一章将展开这些组件如何分层工作，尤其是 CLAUDE.md、Hooks、Skills、Plugins 与 MCP/CIAI 怎样协同。

2 Harness 组件如何分层工作

上一章已经把 Harness 作为 Claude Code 外围的工程环境讲清楚：它由上下文文件、自动化触发器、可复用流程、团队分发包、语言服务、外部系统接口和子代理等部分共同组成。本章进入更细的一层：这些组件各自放在什么位置、解决什么工程问题、什么时候会失效。讲者的核心线索很直接：大型代码库里的 AI 编程能力需要两类条件同时成立：模型足够强，工程组织也能把信息、流程和权限放到合适的层级。

2.1 CLAUDE.md：全局规则与模块级规则

CLAUDE.md 是最基础的一层，因为它在 Claude Code 与项目交互时会进入上下文。根目录的 CLAUDE.md 适合放全局规则：代码风格、禁止事项、验证习惯、权限边界、仓库级构建命令、团队约定。模块级（子目录级）CLAUDE.md 适合放局部规则：某个支付模块的幂等约束、API 模块的错误码规范、风控模块的审计字段、前端模块的组件库约束。

这个分层的工程角色可以用一个简单目录看出来：

```

1 repo/
2   CLAUDE.md
3   services/
4     payment/
5       CLAUDE.md
6       src/
7         refund.ts
8     api/
9       CLAUDE.md
10      src/
11        routes.ts

```

Listing 1: 分层 CLAUDE.md 的典型放置方式

当任务落到 `services/payment/src/refund.ts` 这类路径时，Claude Code 可以先读取根目录规则，再合并支付模块下的局部规则。若把所有规则都塞进根目录文件，支付、风控、API、前端的细节会一起进入上下文；模型正在改退款逻辑时还被迫读到无关前端组件约束，注意力和 token 都会被稀释。分层后的目标是让“总章程”保持短，让“部门手册”只在进入该部门时生效。

可以把有效上下文写成一个近似公式：

$$I(p) = C_{\text{root}}(R) \cup C_{\text{module}}(p) \cup C_{\text{task}}$$

其中各符号含义如下：

本段额外使用的 p 、 C_{task} 与 $I(p)$ 只服务于分层上下文公式。

- R 表示仓库根目录。
- p 表示当前任务触达的文件路径或模块路径。
- $C_{\text{root}}(R)$ 表示根目录 CLAUDE.md 中的全局约束。
- $C_{\text{module}}(p)$ 表示与路径 p 匹配的模块级 CLAUDE.md 约束。
- C_{task} 表示用户本次任务中的直接要求。
- $I(p)$ 表示 Claude Code 在当前路径上真正应当遵守的指令集合。

分层 CLAUDE.md 的设计原则

根目录只放跨模块、长期稳定、必须总是遵守的规则；子目录放本模块独有的业务约束、命名规范、验证命令和风险提醒。这样做的价值很朴素：让 Claude Code 在正确的地方读正确的规则。

常见失败模式也很明确。第一，根目录文件膨胀成“万事通手册”，结果每次对话都背着一大包无关规则。第二，模块级文件写得过细，甚至包含临时 bug 背景或个人偏好，几周后就变成误导。第三，根规则和模块规则互相冲突，例如根目录要求所有服务统一用 `pnpm test`，某个模块文件又要求只跑 `npm test`，Claude Code 会在执行前浪费时间判断谁优先。第四，把密钥、内部地址、未脱敏客户信息写进 CLAUDE.md，这会把配置文件变成泄露入口。

⁴视频画面时间区间：00:05:35–00:06:30

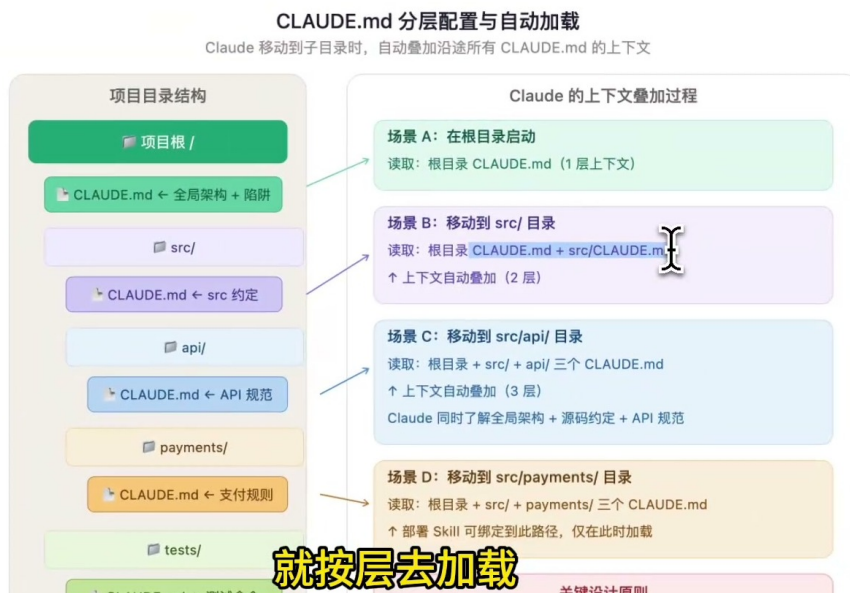


图 4: CLAUDE.md 分层配置与自动加载示意图，展示根目录规则与模块目录规则如何随任务路径叠加进入 Claude Code 上下文。⁴

2.2 Hooks: 把稳定动作变成自动触发

Hooks 的角色是把“每次都该做、靠人提醒容易漏”的动作变成事件触发。它更像 CI 里的小型本地管线：对话结束后整理经验，工具调用前检查权限，编辑后跑格式化，提交前执行测试，读取敏感路径前弹出审批。它的核心价值在于稳定性，因为触发条件由事件控制，行为由脚本或动作定义，减少了临场发挥。

讲者特别提到一个自净化场景：每次对话结束后，系统可以判断本轮对话已经积累到多少轮，达到阈值后再总结经验。这个阈值很关键。每轮结束都总结会浪费 token，也会把太窄的局部经验写成通用规则；隔一段时间总结一次，经验样本更厚，写出的规则更有复用价值。比如设置“每 10 轮对话后总结一次”，再把稳定结论写入团队约定或记忆系统，就比“每说两句话总结一次”更接近工程实践。

Hooks 与 CLAUDE.md 的边界

CLAUDE.md 提供静态规则，Hooks 提供事件动作。前者像项目手册，后者像门禁和流水线；一个告诉 Claude Code 应该遵守什么，一个在关键时刻自动执行检查、整理或阻断。

Hooks 的失败模式通常来自触发条件和副作用。触发太频繁会拖慢交互，让每个任务都背上额外等待时间；触发太宽会把“读文件”这类低风险动作也包上复杂审批；脚本没有幂等性时，重复触发可能反复改同一份配置；自动总结若没有过滤规则，会把一次临时排障的权宜办法写进长期知识；日志处理不当时，Hook 还可能把 token、工单内容或客户字段打印到可见日志里。硬话说清楚：Hooks 一旦写坏，它会把偶发错误变成自动化错误，而且传播速度更快。

Hook 自改进的危险点

自动总结只能沉淀稳定经验，不能把一次性上下文、临时绕过、未验证猜测写成全局规则。更稳的做法是让 Hook 先生成候选总结，再由人或后续校验流程决定是否纳入长期文件。

2.3 Skills：按需加载的专业流程

Skills 解决的是另一个问题：专业任务需要专门流程，但这些流程没有必要总是占用上下文。安全审查、性能剖析、字幕清洗、LaTeX 编译、PR 评论处理，都有相对固定的步骤、检查点、工具偏好和失败处理方式。把它们写成 Skill 后，Claude Code 只有在用户任务匹配触发描述时才加载该流程。



图 5: Skills 的渐进式披露示例：安全审查、文档更新、部署流程与路径绑定只在相关任务触发时加载，减少上下文占用。⁵

这里的关键词是按需加载，也就是 progressive disclosure。Claude Code 先看 Skill 的名称和触发描述；命中后再读 SKILL.md；如果 SKILL.md 指向脚本、模板或样例，再继续读取必要资源。这样既能减少上下文噪声，也能让专业流程保持可维护。安全审查就是典型例子：由安全工程师把入口点枚举、数据流追踪、权限边界、攻击路径验证、误报排除写成 Skill，普通开发者发起“做安全审查”时，Claude Code 就能按这套流程执行，而无需每次让用户重新讲审查标准。

Skill 的工程角色

Skill 是可复用的专业操作手册。它把“怎么做这类任务”从一次性提示词里抽出来，变成可版本化、可测试、可分发的流程资产。

Skills 与 CLAUDE.md 容易混淆。最实用的区分是：长期总是有效的仓库规则进 CLAUDE.md；只有某类任务发生时才需要的步骤进 Skill。例如“禁止直接删除文件”适合放根目录规则；“如何把 Bilibili 讲座转成带图 LaTeX 笔记”适合放 Skill。前者像安全红线，后者像专项工艺卡。

Skills 的失败模式也很现实。触发描述写得太宽，会让无关任务误加载庞大流程；写得太窄，真正需要时又触发不了。Skill 内部如果只写愿景，不写命令、检查点和输出格式，Claude Code 仍然只能临场猜。还有一种隐蔽失败：Skill 引用了脚本或模板，但团队升级路径后没有同步更新，模型会按旧路径执行，直到运行时才暴露问题。最可靠的 Skill 应当包含触发条件、输入边界、执行步骤、验证命令和失败回退。

2.4 Plugins：团队级分发方式

当 Skills、Hooks、MCP 配置和子代理越来越多，单个项目手工复制就会失控。Plugins 的角色是团队级分发：把一组能力打包、版本化、安装、启用、禁用和升级。讲者用 Java 私服和 NPM 管

⁵ 视频画面时间区间：00:08:30–00:09:05

理类比这个层级很贴切：如果公司依赖每个开发者私下拷贝脚本来保持一致能力，版本、来源和升级都会失去治理。

一个团队插件可以包含这些内容：安全审查 Skill、代码迁移 Skill、提交前 Hook、内部工单 MCP 连接配置、专门负责依赖升级的子代理说明、统一的权限声明。新成员加入项目后安装插件，就能获得同一套流程；老项目升级插件，就能同步修复流程缺陷。这件事的价值在大团队里非常硬：它把“口口相传的做法”转成“可发布的工程资产”。

Plugins 与 NPM 类比的边界

NPM 主要分发代码包，Plugins 分发的是 AI 编程环境能力包。二者都需要版本、依赖、发布源和升级策略；Plugins 额外要关注工具权限、触发规则、外部系统凭证和团队审计。

Plugins 的失败模式往往带有组织风险。版本漂移会让不同开发者的 Claude Code 行为不一致；插件过大，会把不相关团队的 Skills 和 Hooks 一起装进来；权限声明过宽，会让一个本来只需读工单的插件获得写生产系统的能力；插件来源没有校验，会带来供应链风险；升级没有变更说明，会让团队无法判断某个 Hook 行为变化来自哪里。真实工程里，插件需要像内部包一样治理：来源可信、版本可追踪、权限最小化、变更可回滚。

2.5 MCP 与 CIAI：连接外部业务系统

MCP 处在 Harness 的外部系统连接层。它让 Claude Code 能通过标准化工具访问业务系统，例如工单平台、测试管理系统、运维系统、知识库、代码托管平台。讲者给的例子很具体：测试人员或业务人员提交工单，里面有标题、描述、图片、复现信息；AI 工具通过 MCP 拉取未处理工单，读取描述和图片，定位相关代码，修复后再把结果推送到运维或工单系统。这样一来，开发流程从“人复制工单信息给 AI”变成“AI 在授权范围内直接读取业务入口”。



图 6: MCP 服务器作为连接内部系统的桥梁，让 Claude Code 在授权范围内访问工单、CI/CD、知识库等外部资源，并进入更完整的自动化工作流。⁶

这个层级的工程价值可以拆成三步。第一，信息入口标准化：工单标题、描述、截图、状态、优先级由系统提供，减少人工转述误差。第二，操作链闭环：Claude Code 读工单、改代码、运行验证、回写处理结果，每一步都可以留下记录。第三，权限可以集中治理：哪些系统可读、哪些字段可写、哪些动作需要审批，都可以在 MCP 服务或企业网关里控制。

CIAI 可以理解为企业自建的 AI 协作与自动化入口。讲者把它与 MCP 并列，是因为很多公司会通过飞书、钉钉、内部运维平台、企业流程引擎来承载 AI 自动化。MCP 更偏协议和工具连接；CIAI 更偏企业流程、协作入口和组织级工作流。对开发团队来说，关键问题落在链路质量上：它是否把“问题来源、上下文读取、代码修改、验证反馈、业务回写”连成可审计闭环。

⁶ 视频画面时间区间：00:09:55–00:10:30

MCP/CIAI 的核心判断

外部系统连接层让 Claude Code 从代码编辑器里的助手，升级为能参与业务流程的工程代理。它带来的收益是闭环自动化；它引入的风险是权限、审计、数据边界和错误回写。

这一层的失败代价最高。工单字段结构变化会导致解析错位；截图或附件权限不足会让 AI 缺少关键证据；回写接口没有幂等保护，重复执行可能多次关闭同一张工单；状态机没有校验，AI 可能把“待验证”误写成“已完成”；读写权限没有分离，原本只需读取工单的能力可能被扩大到修改生产配置；外部系统超时或限流时，如果没有重试与退避策略，自动化流程会卡在半路。更棘手的是数据合规：客户截图、内部日志、缺陷描述可能包含敏感信息，MCP/CIAI 的接入必须有脱敏、审计和授权边界。

外部集成的最低工程护栏

接入 MCP 或 CIAI 前，应先定义只读范围、写入范围、审批动作、审计日志、失败重试、幂等键和敏感字段处理。缺少这些护栏，自动化越顺滑，事故半径越大。

2.6 本章小结

本章把 Harness 的几个关键组件按职责拆开：CLAUDE.md 管长期上下文，根目录放全局规则，模块级规则放局部约束；Hooks 管事件触发，把稳定动作做成自动检查、总结或阻断；Skills 管专业流程，让安全审查这类任务按需加载、按步骤执行；Plugins 管团队分发，把 Skills、Hooks、MCP 配置和子代理打包成可版本化能力；MCP 与 CIAI 管外部业务系统连接，把工单、运维、协作平台纳入 Claude Code 的工作闭环。

这一层的共同原则是“按层放置”。全局规则放全局，模块级规则放模块，专项流程放 Skill，组织复用放 Plugin，业务系统连接放 MCP/CIAI。放错层级就会出现具体问题：上下文膨胀、触发混乱、流程漂移、版本不一致、权限扩大、审计缺失。对大型代码库来说，Harness 的质量决定了 Claude Code 是偶尔可用的聊天助手，还是能稳定进入工程流水线的开发代理。

下一章转入搜索层，说明 CODE_MAP.md 如何把这种分层原则落实到代码定位路径上。

3 Code Map: 让 Claude Code 找得到代码

3.1 默认搜索流程的成本

在大型代码库里，Claude Code 接到一个任务以后，真正困难的地方常常出现在“该看哪里”。如果任务是“修一个支付相关的 bug”，模型先读取根目录里的 CLAUDE.md，理解项目的全局约束、危险操作、代码约定和大致结构；接着它需要在文件系统里寻找相关目录，打开候选文件，判断这些文件和当前任务有没有关系；然后再阅读源代码，学习业务逻辑，最后才进入修改代码这一步。

这个流程里，最昂贵的部分通常发生在定位阶段。模型需要先回答两个问题：当前任务允许做什么，以及相关代码在哪里。前者由根目录 CLAUDE.md 提供边界；后者如果缺少明确导航，就只能依靠目录名、全文搜索、文件列表和逐个打开文件来推断。代码库越大，这个推断过程越像在仓库里逐箱翻找零件：每打开一个箱子都要花时间，每读一段无关代码都要消耗上下文窗口。

默认搜索会消耗上下文预算

默认搜索的风险并非只在“慢”。更直接的工程风险是模型会把上下文预算花在无关文件上，随后在真正相关的文件前已经失去耐心或空间。对 AI 编程工具来说，token 预算就像维修现场的工作台面积：无关零件摆得越多，关键零件越容易被挤到边缘。

可以把一次任务的搜索成本粗略写成下面的关系：

$$S_{\text{map}}(T) \ll S_{\text{full}}(T)$$

- T 表示一次具体开发任务，例如“修复支付回调里的重复扣款问题”。
- $S_{\text{full}}(T)$ 表示缺少索引时，为任务 T 做广泛文件扫描的成本，包含时间、token、打开文件次数和误入无关模块的概率。
- $S_{\text{map}}(T)$ 表示先读取 `CODE_MAP.md`，再进入候选模块后的搜索成本。
- $\ll$ 表示“远小于”。它表达的工程目标是：带索引的搜索应显著少于全量摸索，而非只带来一点点整理感。

这里的公式不追求精确计量。它提醒读者，Code Map 的价值在于改变搜索路径：先用一份小而集中的地图判断方向，再把上下文预算花到最可能相关的模块里。

3.2 索引类比：从全量扫描到模块定位

演讲里用了数据库索引的类比。假设数据库里有一张百万行甚至千万行的表，如果没有索引，查询条件来了以后，数据库只能从头到尾检查大量记录。这个过程叫全表扫描。它能找到结果，但代价很高，尤其在查询频繁或数据规模继续增长时，成本会迅速放大。

代码库也有类似问题。没有 `CODE_MAP.md` 时，Claude Code 看到“支付 bug”这个任务，可能依次检查 `auth/`、`payments/`、`billing/`、`orders/`、`notifications/`、`infra/` 等目录，边打开边猜：“这里和支付有关吗？”“账单模块算不算？”“订单模块会不会也参与？”每一次猜测都要读目录、看文件、比较语义。即使最后找到了 `payments/`，前面那些无关阅读已经消耗了上下文。



图 7：无 `CODE_MAP.md` 与有 `CODE_MAP.md` 的代码搜索路径对比⁷

有了 CODE_MAP.md，搜索会先经过一层轻量索引。这个文件通常不需要复杂格式，只要让模型快速看懂“业务模块”和“代码位置”的对应关系即可。例如：

```
1 # Code Map -- 业务模块索引
2
3 - auth/          认证、授权、OAuth、JWT、权限校验
4 - payments/      支付处理、支付回调、退款、扣款状态
5 - billing/       账单、订阅、发票、周期计费
6 - orders/       订单生命周期、购物车、履约状态
7 - notifications/ 消息通知、邮件、短信、站内信
8 - infra/        基础设施、部署、日志、监控
```

Listing 2: CODE_MAP.md 的典型信息形态

这段内容的关键不在格式漂亮，关键在“可用于定位”。当任务写着“支付回调重复扣款”，模型读完索引后可以优先进入 payments/，同时知道 billing/ 可能只在账单结算阶段相关，notifications/ 多半属于后续通知。索引把“到处打开看看”压缩成“先进入高概率模块，再按需要扩展到邻近模块”。

Code Map 与数据库索引的相似性

数据库索引用小份额外结构换取查询速度；CODE_MAP.md 用小份 Markdown 维护成本换取代码搜索速度。两者都在做同一件事：先建立粗粒度定位，再减少大规模扫描。索引越贴近真实业务边界，搜索路径越短；索引越陈旧，模型越容易被带到错误目录。

3.3 CODE_MAP.md、根目录 CLAUDE.md 与子目录 CLAUDE.md 的协同

CODE_MAP.md 单独存在还不够。演讲强调，要在根目录 CLAUDE.md 里明确告诉 Claude Code：当它需要理解代码结构或定位模块时，先阅读 CODE_MAP.md。这一步非常具体，因为根目录 CLAUDE.md 是会话开始时稳定进入上下文的文件；只有在那里写清楚，模型才会把 Code Map 当作默认导航入口。

推荐的三层路径如下：

1. 第一层读取根目录 CLAUDE.md：获得全局架构说明、禁止事项、代码约定，以及“搜索代码前先读 CODE_MAP.md”这条导航规则。
2. 第二层读取 CODE_MAP.md：把任务映射到业务模块或目录，例如从“支付 bug”映射到 payments/，必要时标记 billing/ 或 orders/ 为相邻模块。
3. 第三层进入目标模块后读取模块内 CLAUDE.md：获得本模块的测试命令、关键文件、业务规则、接口约束和容易踩坑的位置。

这个流程的好处是边界逐层收紧。根目录 CLAUDE.md 管全局约束，CODE_MAP.md 管模块定位，模块 CLAUDE.md 管局部规则。三者各自承担一类问题，Claude Code 不需要在一个超长说明文件里同时寻找全局禁令、业务地图和模块细节。

⁷ 视频画面时间区间：00:12:20–00:13:05

⁸ 视频画面时间区间：00:14:45–00:15:45



图 8: 从根目录规则到 CODE_MAP.md 再到模块规则的分层检索结构⁸

三层导航顺序

实用的导航顺序可以压缩成一句话: 先用根目录 CLAUDE.md 决定搜索规矩, 再用 CODE_MAP.md 决定去哪个模块, 进入模块后再读模块 CLAUDE.md 决定怎样改。这个顺序让模型从“项目级方向”走到“模块级操作”, 减少迷路和误改。

维护 CODE_MAP.md 时, 应把它写成“任务可用的索引”, 而非写成组织架构图。模型需要知道“处理支付回调要看哪里”“账单周期计费在哪个目录”“订单状态机在哪些文件附近”。如果新增了模块、新增了核心接口, 或者某个目录的职责发生迁移, Code Map 要同步更新。否则它会像过期地图一样, 把模型带到旧入口。

3.4 模型升级后的配置复查

演讲最后提醒了一个容易被忽视的维护点: 模型升级后, 需要复查 CLAUDE.md、CODE_MAP.md 和模块级规则。原因很现实, 模型能力会变化。旧版本需要明确约束的行为, 新版本可能已经内置得更稳; 旧版本经常误判的任务, 新版本可能通过更好的代码理解能力解决; 团队曾经写进配置里的补丁式规则, 可能在升级后变成噪声。

这并不意味着配置会自动变少。更准确地说, 配置文件要跟着模型能力重新校准: 已经被模型稳定掌握的低价值规则可以压缩; 仍然属于本项目特有业务边界、合规要求、危险操作和模块导航的信息必须保留; 新版本引入的新能力如果改变搜索或执行方式, 也要把对应规则补进去。

模型升级后的配置漂移

升级后的配置复查如果缺席, 团队会遇到两类问题: 一类是过期规则继续占用上下文, 模型每次都读到低价值噪声; 另一类是新模型的能力边界已经移动, 旧配置没有覆盖新的使用方式。大型代码库里的 AI 配置应当像依赖版本一样管理, 有复查周期, 有负责人, 有删改记录。

一个务实的复查周期是每次重要模型发布后检查一次; 如果发布节奏很快, 至少每三到六个月安排专人复查。复查时可以用几组真实任务做回归: 支付修复、账单变更、认证改造、通知链路排查。观察模型是否仍会走错目录, 是否仍需要某条规则提醒, 是否能稳定先读 CODE_MAP.md。这些

结果比抽象讨论更可靠。

3.5 本章小结

本章的核心是让 Claude Code 在大型代码库里先找到方向。默认搜索流程会读取根目录 `CLAUDE.md`，再通过文件系统和源码逐步判断相关模块；当代码库规模上来后，这种方式会消耗大量 token 和时间，还会提高误入无关模块的概率。

`CODE_MAP.md` 的角色是代码库索引。它把业务模块、目录和关键职责摆在一页里，让模型先做模块定位，再深入阅读代码。根目录 `CLAUDE.md`、`CODE_MAP.md`、模块 `CLAUDE.md` 形成一条稳定导航链：全局规则负责边界，Code Map 负责定位，模块规则负责局部执行。

最后，AI 配置要随模型升级复查。模型能力变强后，某些旧约束会变成噪声；项目特有的业务边界、危险操作和模块导航仍然需要保留。对团队来说，维护 Code Map 和 `CLAUDE.md` 的价值在于把模型的搜索路径固定到可控、可检查、可持续的工程流程里。

下一章把这些配置压缩成落地顺序，说明先交付哪些基础入口，再推进自动化、分发和外部集成。

4 实施路线图

前面几章已经把 Harness 的组成、分层上下文、代码地图和搜索成本讲清楚。收尾处，讲者把这些组件压缩成一条可执行路线：先让 Claude Code 能读到稳定的项目规则和代码索引，再把高频动作沉淀成自动化与技能，最后再进入团队分发、外部系统连接和子代理编排。这个次序很现实，因为大型代码库里的改造失败，常见原因是基础入口混乱：规则散在聊天记录里，代码地图缺失，目录级约束靠人肉提醒，结果模型每次都要重新摸索项目。

讲者在路线图之前还补了一句维护要求：这类配置应当每隔 3–6 个月复查一次，并安排专人负责。这个提醒很关键。Harness 会随着模型能力、项目结构、团队流程和外部工具变化而老化；如果没人定期检查，早期写下的规则可能会从“节省上下文”变成“误导上下文”。



图 9: Harness 构建优先级金字塔与实施路线图⁹

⁹ 视频画面时间区间：00:16:45–00:17:12

这张图表达的是优先级和依赖关系。底层的 CLAUDE.md/CODE_MAP.md/模块级规则、Hooks 自净化、Skills 工作流沉淀可以并行推进，因为它们都属于基础能力；Plugins、MCP 和 Subagents 更适合在这些基础能力稳定后再扩展。

4.1 第一阶段：CLAUDE.md、CODE_MAP.md、模块级规则

第一阶段要交付三件基础产物：根目录的 CLAUDE.md、代码库地图 CODE_MAP.md，以及关键模块目录下的模块级 CLAUDE.md。讲者把它们放在优先级最高的位置，因为它们决定 Claude Code 进入项目后的第一视角。

根目录 CLAUDE.md 承担全局约束：项目使用什么语言和构建工具，哪些命令可以验证改动，哪些目录需要谨慎处理，哪些团队规则必须始终遵守。它像办公楼门口的总平面图和安全须知，模型一进门就要看到。

CLAUDE.md 可以手写，也可以先用 Claude Code 的初始化命令 `/init` 生成初稿，再由团队补充项目约束、常见陷阱、验证命令和模块入口。初稿的价值是快速启动；真正可靠的版本仍要靠团队把项目特有知识补进去。

CODE_MAP.md 在这里作为第一阶段交付物出现，沿用上一章“代码库索引”的定义，并负责把任务 T 导向候选模块。

这里可以用一个简单关系表达路线图的第一步：

$$C_{\text{root}}(R) + M + C_{\text{module}}(p) \implies E_{\text{project}}(T)$$

其中：

- C_{root} 表示根目录全局上下文文件 CLAUDE.md；
- M 表示代码地图 CODE_MAP.md，用来把任务导向更可能相关的模块；
- C_{module} 表示子目录级规则，也就是模块内部的局部约束；
- $E_{\text{project}}(T)$ 表示任务 T 进入项目后的可执行入口状态：有明确入口、稳定边界和可复查依据。

模块级规则的作用更局部。比如支付模块、权限模块、前端组件库、数据迁移目录，往往各自有独立禁忌和验证方式。把这些规则写在对应目录下，可以让 Claude Code 在进入模块时读取更贴近现场的约束。全局规则解决“这个项目总体怎么做”，模块级规则解决“这个模块具体怎么做”。

第一阶段的交付标准

第一阶段完成后，一个新任务至少应能回答三个问题：先看哪个文件、进入哪个模块、用哪个命令验证。若这三个问题仍靠人工在对话里反复补充，基础层还没有建好。

4.2 第二阶段：Hooks 与 Skills 的沉淀

第二阶段开始处理重复动作。讲者提到，下一步可以写 Hooks，并进一步把 Hooks 做成能够持续改进的机制；同时也可以做格式化、一体化流程等稳定动作。这里的重点是把“每次都要提醒模型做的事”变成事件触发。

Hooks 适合承接确定性强的动作。例如：写完文件后运行格式化，提交前检查禁止命令，任务结束时记录失败原因，检测到特定目录改动后执行对应测试。它们的价值来自稳定触发，并减少对语言模型临场判断的依赖。越是规则明确、输入输出清楚、失败条件可检测的动作，越适合放进 Hooks。

讲者还提到 Skills：把工作中常用的工作流沉淀起来。Skills 更像“可按需调用的操作手册”。例如生成代码审查报告、处理字幕、构建 PDF、排查 CI、整理知识库，这些任务通常有固定步骤、固定检查项和固定产物。写成 Skill 后，模型只在相关任务中加载它，避免把所有流程长期塞进上下文。

Hooks 与 Skills 的直观区别

Hooks 像门禁刷卡后的自动记录，事件发生就触发；Skills 像工具柜里的专用说明书，任务需要时再取出来读。前者强调自动执行，后者强调按需加载的专业流程。

这一步的风险也更工程化：Hooks 如果能写文件、运行命令或连接外部系统，就必须有权限边界和日志；Skills 如果写得太宽泛，会把任务重新拖回冗长提示词。好的第二阶段改造，应当从最频繁、最稳定、最容易验证的流程开始，别一上来追求覆盖所有场景。

第二阶段的常见失败点

把尚未稳定的团队习惯提前写进 Hooks，会放大错误；把一次性经验包装成 Skills，会制造维护负担。先沉淀每周都发生、结果可以检查、失败后能追踪的流程。

4.3 第三阶段：Plugins、MCP 与 Subagents

当基础文件、代码地图、Hooks 和 Skills 都有了稳定版本，第三阶段才适合推进 Plugins、MCP 和 Subagents。讲者的意思很清楚：团队级分发和外部集成需要前面的基础产物支撑。否则插件只是把混乱打包，MCP 只是把混乱接到更多系统，子代理只是把混乱并行化。

Plugins 适合团队复用。它可以把 Skills、Hooks、MCP 配置和相关约定一起分发，让多个项目或多名成员共享同一套工作方式。适用场景包括：团队统一代码审查流程、统一安全扫描步骤、统一文档生成流程、统一发布前检查。

MCP 负责把 Claude Code 接入外部业务系统，例如工单、知识库、内部工具或数据源。它带来的收益是减少人工复制粘贴，让模型能在授权范围内读取上下文或写回结果。这里必须把权限、审计和最小授权讲清楚：外部系统越重要，权限范围、操作记录和失败回滚要先写清楚；仅验证连接成功远远不够。

Subagents 适合拆分复杂任务。大型代码库中的调研、实现、测试、文档、图像选择、最终一致性检查，常常可以由不同子代理并行处理。前面的章节已经说明，子代理的价值来自明确分工和边界；到了第三阶段，它们可以接入成体系的上下文文件、代码地图和 Skills，从而减少各自乱搜、乱写、重复定义的问题。

第三阶段的准入条件

Plugins、MCP 与 Subagents 应当在基础层可复查之后推进。若 CLAUDE.md、CODE_MAP.md、Hooks 和 Skills 尚未稳定，团队级分发会把局部问题扩散到更多仓库和更多人。

4.4 本章小结

这一路线图的核心是先建入口，再沉淀动作，最后扩展组织级能力。第一阶段交付 CLAUDE.md、CODE_MAP.md 和模块级规则，让 Claude Code 知道从哪里开始、哪些边界不能越过、哪些模块更相关。第二阶段把重复且可验证的动作变成 Hooks，把高频工作流写成 Skills。第三阶段再做 Plugins、MCP 和 Subagents，让团队分发、外部系统连接和并行任务处理建立在稳定基础之上。

讲者给出的 3-6 个月复查周期，也应当被视为路线图的一部分。Harness 是活的工程配置，模型升级、项目重构、团队流程变化都会改变它的最佳形态。没人维护的 Harness 会逐渐失真；有人负责复查，它才会持续节省时间和上下文。

5 总结与延伸

5.1 讲者的收束观点

讲者最后保留了几个实质判断。第一，这篇文章里提到的知识非常实用，值得拿到真实项目里深入实践；只有在自己的代码库中跑过一轮，才会体会到上下文文件、代码地图、Hooks、Skills 和外部集成分别解决什么痛点。第二，这些做法可以迁移到不同 AI 编程工具里。原因很直接：只要项目变大，工具都会面对相似问题，包括代码定位成本、上下文噪声、团队规则缺失、重复流程难以固化、外部系统上下文割裂。第三，视频没有覆盖所有细节，后续应继续查阅官方文档，把遗漏能力补齐。

这个收束把 Claude Code 放在大型代码库的 AI 编程方法论中观察：把项目规则写成稳定文件，把搜索路径做成地图，把重复动作做成自动化，把常用流程做成可复用技能，把团队经验包装成可分发能力。

最终 takeaway

大型代码库里的 AI 编程效率，取决于模型能力，也取决于项目是否提供了清晰入口、可靠索引、可触发流程和可维护集成。Claude Code 只是这套实践的一个具体落点。

5.2 跨工具迁移：为什么这些做法适用于多种 AI 编程工具

跨工具迁移成立，是因为问题主要来自代码库规模，工具品牌更多影响实现细节。仓库越大，文件数量、模块边界、历史约定、测试矩阵和业务依赖就越复杂。任何 AI 编程工具进入这种环境，都会遇到同一组工程摩擦：不知道规则、找错模块、重复搜索、遗漏验证、把局部经验当全局规则。

因此，可迁移的是背后的工程结构，具体文件名只是某个工具里的实现形式：

- 上下文文件对应“入口规则”：让模型先读稳定约束；
- 代码地图对应“导航索引”：让模型先缩小搜索范围；
- Hooks 对应“事件自动化”：让确定性动作不用反复口头提醒；
- Skills 对应“流程资产”：让高频任务有可复用步骤；
- Plugins 对应“团队分发”：让个人经验变成共享配置；
- MCP 与类似集成对应“外部上下文通道”：让工单、知识库、内部系统进入授权 workflow；
- Subagents 对应“任务拆分”：让复杂工作按职责并行推进。

这也是讲者建议继续看官方文档的原因。官方文档能补足版本能力、权限模型、配置格式和最新限制。实践路线图负责告诉读者先做什么；文档负责告诉读者每个工具在当前版本中具体怎么做。

5.3 实践检查清单

下面这份清单可以作为落地时的第一轮验收。它故意写得很具体，因为大型代码库的 AI 改造最怕停留在口号层面。

- 根目录是否已经有 `CLAUDE.md`，并写清构建、测试、格式化、禁止操作和协作规则？
- 关键业务模块是否已经进入 `CODE_MAP.md`，每个模块是否能对应到目录、入口文件和常见修改点？
- 高风险或高频目录是否有子目录级规则，例如支付、权限、迁移、基础组件、生成文件目录？
- 是否记录了模型完成任务后必须执行的验证命令，减少人工在每次对话里的临时补充？
- 是否把格式化、测试选择、禁止命令检查、会话总结等稳定动作沉淀成 Hooks？
- 是否把重复出现的工作流写成 Skills，例如代码审查、PDF 生成、字幕清洗、CI 排障、知识库整理？
- Hooks 是否有日志、失败提示和权限边界，避免自动化静默修改关键文件？
- Skills 是否有明确触发条件、输入要求、输出文件和最终检查项？
- 团队是否需要把这些配置包装成 Plugins，在多个仓库之间统一分发？
- MCP 或其他外部集成是否明确了最小权限、审计记录和失败回滚路径？
- 复杂任务是否可以拆给 Subagents，并为每个子代理规定只读范围、写入文件和交付格式？
- 是否设定了 3-6 个月一次的配置复查责任人，检查规则是否过期、代码地图是否失真、Hooks 和 Skills 是否仍然有效？
- 是否在模型或工具升级后重新验证 Harness，避免默认旧配置继续适用？

如果这份清单只能完成前几项，也值得先做。讲者给出的路线图本来就是分阶段改造：基础入口先稳定，自动化随后沉淀，组织级扩展最后跟上。真正需要警惕的是跳过基础层，直接追求插件、MCP 或多子代理编排；那样只会把搜索混乱、规则缺失和验证不足扩散到更大的范围。